

Machine Learning Model Development

Alka Bhushan

Data Preprocessing and Feature Engineering

- Data cleaning: handling missing values, duplicates, and outliers
- Data transformation: scaling, normalization, and categorical encoding
- Feature engineering: aggregation, temporal, and derived features
- Train/validation/test dataset preparation
- **Scikit-learn Pipelines** for small and medium-sized datasets
- **Spark ML Pipelines** for large-scale distributed preprocessing and feature engineering
- Processed datasets can be stored as **Parquet/Delta Lake** and used by any training framework (Scikit-learn, XGBoost, PyTorch, Transformers)

Popular Machine Learning Frameworks and Libraries

Classical Machine Learning

- Scikit-learn (Single-node CPU)
- XGBoost (CPU/GPU, Distributed Support)
- Spark MLlib (Distributed ML on Spark)

Distributed Training

- PyTorch Distributed (Multi-GPU, Multi-node)
- Ray Train (Distributed Training Orchestration)
- DeepSpeed (Large-scale Model Training)
- BigDL / Orca (Spark-based Distributed AI)

Deep Learning

- PyTorch (CPU/GPU)
- TensorFlow (optional)

LLM Development & Fine-Tuning

- Transformers
- PEFT

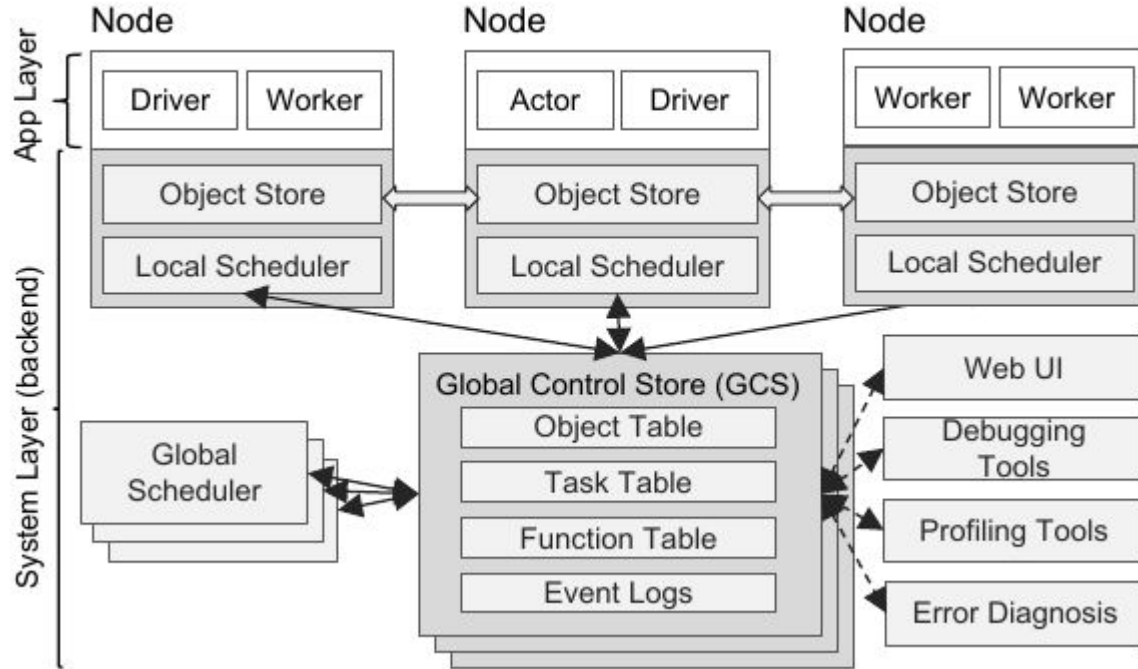
Hyperparameter Tuning

- Ray Tune

Ray: A Distributed Framework for AI Applications <https://github.com/ray-project/ray>

- Published in 2017
- Open-source distributed computing framework and runtime for Python.
- Provides a Python API for distributed execution across clusters.
- Motivated by the need of RL applications
 - Massive computation
 - Millions of simulations and training tasks
 - Large scale distributed execution
 - Heterogenous and dynamic computation
 - Mix of CPU and GPU tasks
 - Computation graph evolves during execution
 - Low latency requirements
 - Tight loop between simulation, serving, and training
 - Milliseconds-level scheduling and data access needed
- Traditional data-processing systems such as Apache Spark were designed primarily for batch analytics and data-parallel processing.

Architecture



Application Layer

Driver

- A process executing user program
- Submits tasks

Workers

- Stateless processes
- Execute remote functions (tasks) invoked by driver or another worker
- Workers are started automatically and assigned tasks by the system layer.
- No state retained between tasks

Actors

- Stateful processes
- Maintain internal state
- Execute methods serially of a class

System Layer

Global Control Store (GCS)

Stores:

- Task metadata
- Object metadata
- Function metadata
- Lineage information

Benefits:

- Fault tolerance
- Scalability
- Debugging support

In Memory Distributed Object Store

Features:

- Shared-memory storage on each node
- Immutable objects
- Apache Arrow format
- Zero-copy data sharing

Benefits:

- High throughput
- Low latency
- Efficient data transfer

The system layer implements task scheduling and data management to satisfy the performance and fault-tolerance requirements.

Distributed Scheduling

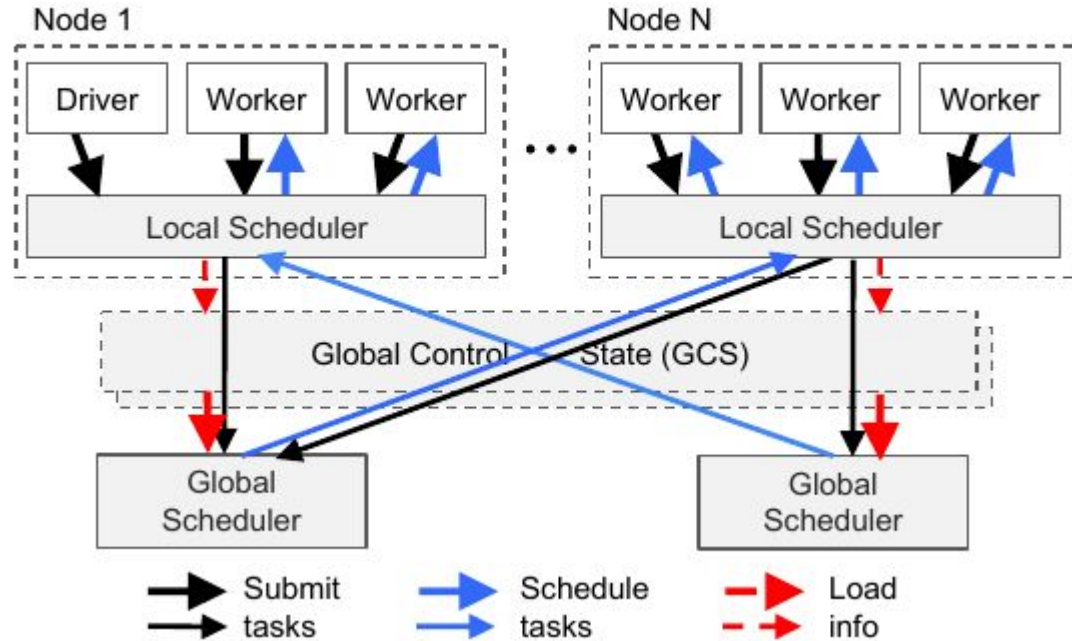
Scheduling Strategy

1. Task submitted to local scheduler
2. Execute locally if resources available
3. Otherwise forward to global scheduler
4. Global scheduler selects node based on:
 - Resource availability
 - Queue length
 - Data locality

Advantages

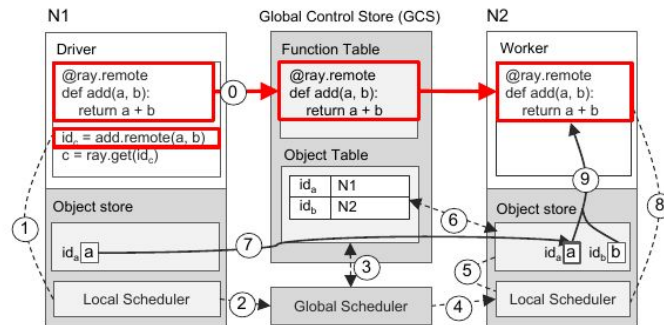
- High scalability
- Low scheduling latency
- Data-local execution
- Supports millions of tasks/sec

Bottom-up distributed scheduler

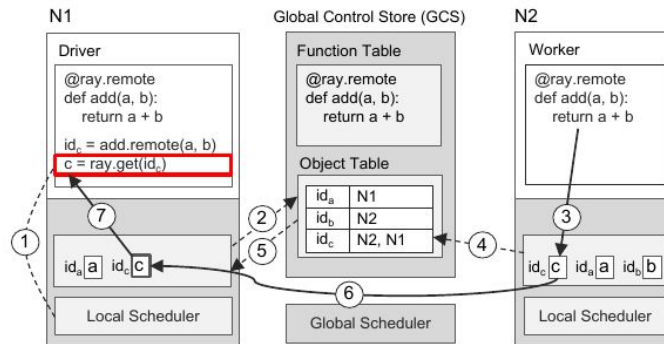


An Example

add two objects a and b, which could be scalars or matrices and return object c



(a) Executing a task remotely



(b) Returning the result of a remote task

Ray AI Libraries

- Data : Scalable datasets for ML
 - **Faster & Cost-Efficient** – Streams data between CPUs and GPUs to maximize utilization.
 - **Framework Friendly** – Integrates with PyTorch, TensorFlow, Hugging Face, and vLLM.
 - **Multi-Modal Support** – Handles Parquet, CSV, JSON, images, audio, and video.
 - **Scalable by Default** – Runs seamlessly from a single machine to large distributed clusters.
 - **Cloud Ready** – Works across AWS, GCP, and Azure.
- Train: Distributed Training
- Tune: Scalable Hyperparameter Tuning
- RLib; Scalable Reinforcement Learning
- Serve: Scalable and Programmable Serving

Ray Data : Datasets and Blocks

- **Dataset:** Distributed collection of data with lazy transformations
- **Block:** Partition of a dataset; basic unit of storage and computation
- **Lazy Execution:** Operations execute only when consumed or materialized
- **Parallel Processing:** Blocks are processed independently across the cluster
- **Efficient Storage:** Blocks stored in Ray Object Store using Arrow/Pandas formats
- **Scalable:** Same code runs from one machine to large distributed clusters

Ray Data: Operators, Plans & Streaming Execution

Two-Phase Planning

- **Logical Plan:** Describes *what* operations to perform
 - Example: Read → Map → Filter → Select
- **Physical Plan:** Describes *how* to execute them
 - Converts logical operators into Ray tasks/actors

Optimization

- Plans are optimized before execution
- **Operator Fusion** combines adjacent transformations to reduce overhead
- One logical operator may become multiple physical operators

Streaming Execution

- Data flows through operators as **streams of blocks**
- Multiple stages execute **concurrently**
- Enables independent CPU and GPU resource allocation
- Improves throughput and resource utilization

Execution Trigger

- Plans are executed only when the dataset is **materialized or consumed**
 - Examples: `show()`, `write_*()`, training/inference

Ray Train

What is Ray Train?

- Scalable library for **distributed training and fine-tuning**
- Runs training seamlessly from **one machine to a cluster**
- Simplifies distributed computing for large models and datasets

Core Concepts

1. **Training Function (train_func)**
 - Contains model training logic
2. **Worker**
 - Process executing the training function
3. **Scaling Configuration**
 - Defines number of workers and CPU/GPU resources
4. **Trainer**
 - Launches workers and manages distributed execution

```
def train_func():  
    """User-defined training function that runs on each distributed  
    worker process.
```

This function typically contains logic for loading the model,
loading the dataset, training the model, saving checkpoints,
and logging metrics.

```
"""  
  
...
```

```
from ray.train import ScalingConfig  
  
# Single worker with a CPU  
scaling_config = ScalingConfig(num_workers=1, use_gpu=False)  
  
# Single worker with a GPU  
scaling_config = ScalingConfig(num_workers=1, use_gpu=True)  
  
# Multiple workers, each with a GPU  
scaling_config = ScalingConfig(num_workers=4, use_gpu=True)
```

```
from ray.train.torch import TorchTrainer  
  
trainer = TorchTrainer(train_func, scaling_config=scaling_config)  
trainer.fit()
```

Ray Tune: Hyperparameter Tuning

- Automates hyperparameter optimization
- Runs multiple training trials in parallel
- Supports **Grid Search**, **Bayesian Optimization**, **ASHA**, **PBT**
- Scales from **single machine** to **distributed clusters**
- Integrates with **PyTorch**, **TensorFlow**, **XGBoost**, **Hugging Face**

```
from ray import tune

def objective(config): # ①
    score = config["a"] ** 2 + config["b"]
    return {"score": score}

search_space = { # ②
    "a": tune.grid_search([0.001, 0.01, 0.1, 1.0]),
    "b": tune.choice([1, 2, 3]),
}

tuner = tune.Tuner(objective, param_space=search_space) # ③

results = tuner.fit()
print(results.get_best_result(metric="score", mode="min").config)
```

Ray Serve: Scalable Model Serving

What is Ray Serve?

- Framework-agnostic library for **online model inference**
- Deploys ML models as scalable APIs
- Supports LLMs, deep learning models, and custom Python logic

Key Features

- **Model Serving & Composition** – Combine multiple models and business logic
- **Autoscaling** – Dynamically adjusts replicas based on workload
- **Efficient Inference** – Request batching and response streaming
- **Resource Aware** – Supports CPU, GPU, and multi-node deployments
- **Framework Agnostic** – Works with PyTorch, TensorFlow, Scikit-Learn, Hugging Face, etc